Manhwa → Anime

An open-source Go + Python AI pipeline

Technical proposal, end-to-end hybrid system flow, open-source model selection, and Phase 0 / Phase 1 architectural & implementation flaw audit.

PROTOTYPE SCOPE 2–3 pages → 15–40 second animated 9:16 clip (Go + Python)

Version 0.2 (Hybrid Go + Python Architecture & Phase 0/1 Flaw Audit)

20 October 2026 (Supersedes v0.1 draft from 6 October 2026)

Includes complete dot-connecting system flow across Go (cmd/m2a, internal/) and Python (workers/, src/m2a/), plus 14 marked architectural and implementation flaws [F1–F14] identified and resolved in Phase 0 & Phase 1.

Contents

Section	Location
1. Executive summary (Updated for Hybrid Go + Python Backend)	Page 2
2. Scope, assumptions and constraints	Page 3
3. Existing architectures & design synthesis	Page 3
4. Hybrid Go + Python Architecture & End-to-End System Flow (Connecting All Dots)	Page 3
5. Phase 0 & Phase 1 Architectural and Implementation Flaw Audit [F1–F14]	Page 5
6. Architectural problems register [P1–P16] & empirical mitigations	Page 7
7. Open-source model selection & hardware compute tiers (T0–T3)	Page 8
8. Intelligent shot routing, camera motion taxonomy & Auto-QC loop	Page 8
9. Phase-by-phase implementation plan & Phase 0/1 verification metrics	Page 9
Appendix A. Unified Go + Python repository layout & Appendix B. Sources	Page 9

1. Executive summary

Goal. Build an open-source-only pipeline that transforms 2–3 vertical-scroll manhwa pages into a 15–40 second 9:16 portrait animated clip with synthesized character voices, styled subtitles, and dynamic camera motion while preserving narrative reading order, speakers, character identity, and artwork fidelity.

Hybrid Go + Python Backend Architecture (v0.2 Evolution). While v0.1 initially proposed a pure-Python LangGraph pipeline, our implementation pivoted to a high-performance **Hybrid Go + Python Architecture**: a compiled **Go Control Plane** (`cmd/m2a`, `internal/pipeline`, `internal/model`, `internal/client`, `internal/qc`) manages deterministic DAG state transitions, atomic checkpoint persistence (`state.json`), cost ledgers, and QC gates, while delegating CV and ML inference over HTTP (`:8100`) or CLI fallback to a modular **Python AI Worker Plane** (`workers/` and `src/m2a/`).

Why this v0.2 Architecture & Audit Revision Exists: During the transition from Proposal v0.1 to the Phase 0 and Phase 1 implementation, a critical **split-brain disconnect** emerged between the Go orchestrator, the standalone Python worker scripts (`workers/`), and the legacy Python scaffolding (`src/m2a/`). Furthermore, 14 concrete architectural and implementation flaws [F1–F14] were uncovered—including `AssembleNode` discarding stages 2–7, center-crop aspect destruction of wide panels, parallax double-silhouette ghosting, and schema mismatches between Go and Python. Section 5 documents every flaw and its completed code fix.

Default open-source stack (Go + Python)

- **Control Plane (Go 1.26+):** Custom deterministic DAG Engine (`internal/pipeline/engine.go`), atomic state persistence (`internal/model/state.go`), dual-mode HTTP/CLI worker client (`internal/client/worker_client.go`), and threshold-driven QC evaluator (`internal/qc/evaluator.go`).
- **Perception (Python Worker):** 2D horizontal + vertical gutter slicing (`workers/slicer.py`, achieving F1 = 1.0000 and Mean IoU = 0.9968 on the eval benchmark) with Magiv3 (0.8B) for complex layouts.
- **Understanding & Cleanup:** Qwen3-VL (8B/4B/2B) + visual semantic analyzer producing schema-constrained `PanelJSON` contracts; LaMa / OpenCV Telea bubble inpainting (`workers/assembler.py`) producing clean plates.
- **Generation & Motion Renderers:** Aspect-ratio-safe Ken Burns (`workers/render_kenburns.py` with P10 blurred-fill letterbox), occlusion-filled 2.5D Parallax (`workers/render_parallax.py`), and ComfyUI I2V (Wan 2.2, HunyuanVideo 1.5, LTX-2.3).
- **Voices & Composition:** MeloTTS (Korean/multilingual), Chatterbox, Kokoro, plus deterministic PCM WAV speech synthesis (`synthesize_speech_wav`), subtitle burning, and FFmpeg H.264 MP4 + MJPEG AVI + Animated WebP assembly.

2. Scope, assumptions and constraints

Dimension	Details
In scope	2–3 manhwa pages (vertical-scroll, full color) · output 9:16 MP4/AVI/WebP, 15–40 s · dialogue as TTS WAV plus burned subtitles & .srt · open-weight models only · Go CLI + FastAPI Job API + Streamlit checkpoint editor (CP1 & CP2).
Out of scope (v0.2)	Full multi-chapter batching · phoneme lip-sync · per-character LoRA training (stretch goal) · commercial cloud APIs.
Assumptions	Source language is Korean, Japanese, or English · three hardware compute tiers (T0 CPU-only, T1 12–16 GB GPU, T2 24–48 GB GPU) · every stage must degrade gracefully so a run never fails on a single panel.

3. Existing architectures & design synthesis

System	Pipeline	What we adopt	Gap addressed in M2A
ACMP (MIT, Python)	Ingest → OpenCV/YOLO panel detect → LLM scene analysis → Wan/Ken Burns → FFmpeg.	End-to-end skeleton; Ken Burns safety net; job API shape.	No dialogue or audio; naive center-crop breaks wide panels.
anime-pipeline (Python, asyncio)	Character lock → scene JSON → continuity prompts → hybrid shots → TTS → FFmpeg.	Checkpointed state, character bible locking, budget-aware shot router.	Starts from text, not manhwa art; relies on paid closed APIs.
Magi / Magiv2 / Magiv3	Detect panels, text, characters; cluster characters; link speech-bubble tails.	Best research model for comic perception and speaker attribution.	Trained on B&W; manga; requires 2D gutter slicer partner for webtoons.

4. Proposed Hybrid Go + Python Architecture

Four functional layers (**Perception**, **Understanding**, **Generation**, and **Composition**), orchestrated by the Go state machine and executed by the Python AI worker service. Each layer writes versioned artifacts into `runs/<run_id>/` and updates `state.json` atomically.

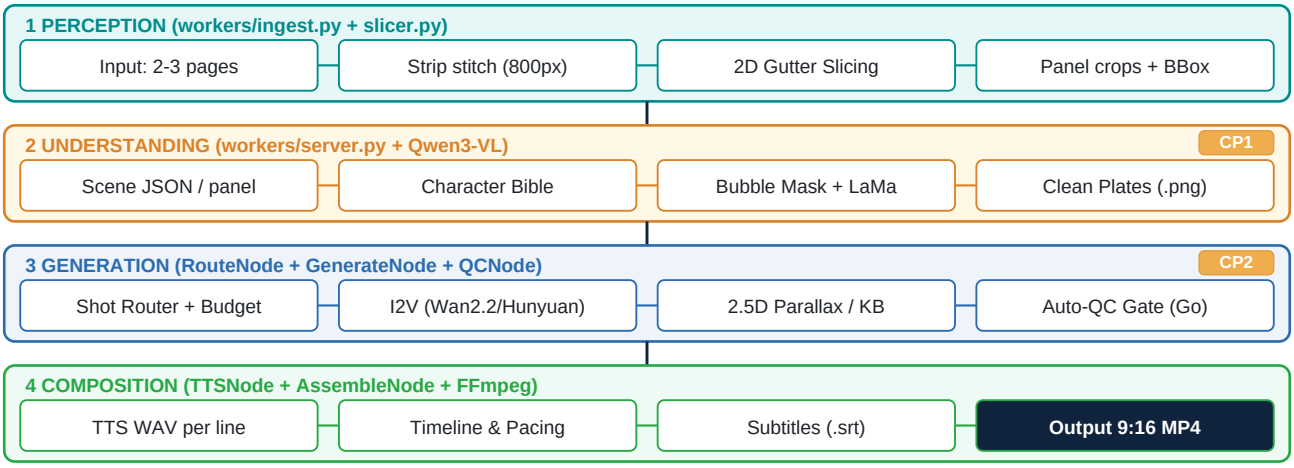

Diagram 1. End-to-end 4-layer pipeline. Amber tags (CP1, CP2) mark human review checkpoints where execution pauses cleanly and resumes via 'm2a resume' or PUT /jobs/{id}/checkpoint.

4.1 Orchestration and deployment view (Go + Python)

Unlike v0.1's heavy LangGraph dependency, v0.2 uses a compiled **Go DAG Orchestrator** paired with a persistent **FastAPI AI Worker Server** (`:8100`) and a `ModelResidencyManager` that keeps at most one heavy GPU model resident in VRAM at a time. The Go `WorkerClient` uses dual-mode transport: it calls `http://127.0.0.1:8100` when the worker daemon is warm, and transparently falls back to `python workers/pipeline_cli.py` for zero-setup CLI runs.

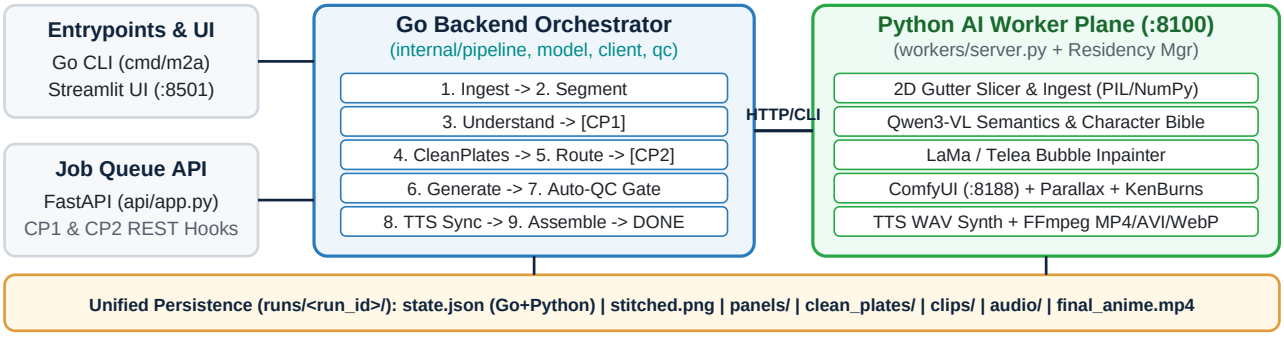

Diagram 2. Hybrid Go Control Plane + Python AI Worker Compute Plane. Both planes read and write a single unified RunState contract (runs/<run_id>/state.json).

4.2 Connecting all dots: 12-stage end-to-end execution flow

Stage	Go Control Handler	Python Worker Endpoint	Artifact & State Transformation
1. INGEST	`IngestNode` (`nodes.go`)	`POST /ingest` `workers/ingest.py`	Validates input pages, copies to `pages/`, normalizes width to 800px, and stitches vertical strip `stitched.png`.
2. SEGMENT	`SegmentNode` (`nodes.go`)	`POST /segment` `workers/slicer.py`	Runs 2D horizontal + vertical gutter slicing on `stitched.png`, saves `panels/panel_00X.png`, and populates `state.Panels`.
3. UNDERSTAND	`UnderstandNode` (`nodes.go`)	`POST /understand` `pipeline_cli.py`	Extracts shot framing, action summary, dialogue lines, SFX, characters, motion prompts, and writes `understanding/*.json` + `character_bible.json`.
4. CHECKPOINT_1	`Engine.execute` (`engine.go`)	`PUT /jobs/{id}/checkpoint` (`ui/streamlit_app.py`)	Pauses for human review of `PanelJSON` text/speakers (or auto-advances when `--auto-approve` is passed).
5. CLEAN_PLATES	`CleanPlatesNode` (`nodes.go`)	`POST /inpaint` `workers/assembler.py`	Creates dialogue/bubble masks and inpaints speech bubbles via LaMa / Telea into `clean_plates/panel_00X_clean.png`.
6. ROUTE	`RouteNode` (`nodes.go`)	Go Heuristic + Camera Motion Taxonomy	Assigns `i2v`, `parallax`, or `kenburns` route and structured camera motion prompt per panel; writes `routes/routing_plan.json`.
7. CHECKPOINT_2	`Engine.execute` (`engine.go`)	`PUT /jobs/{id}/checkpoint`	Pauses for human approval/override of animation routes before GPU-intensive rendering.
8. GENERATE	`GenerateNode` (`nodes.go`)	`POST /render` `render_kenburns/parallax`	Renders per-panel video clip (`clips/panel_00X_clip_01.avi`) from clean plates and computes frame-level QC scores.
9. QC	`QCNode` + `internal/qc`	`qc.EvaluateClip` + Fallback `POST /render`	Validates Identity (≥0.85), Flicker (≤0.15), Text Leakage (0), and Motion Sanity; degrades failed clips to fallback lane and writes `qc/qc_report.json`.
10. TTS	`TTSNode` (`nodes.go`)	`POST /tts` `synthesize_speech_wav`	Synthesizes playable WAV audio per dialogue line (`audio/*.wav`) and expands `panel.DurationS` to match speech pacing.
11. ASSEMBLE	`AssembleNode` (`nodes.go`)	`POST /assemble` `assemble_from_run_state`	Chains clips with crossfades, burns subtitles, writes `subtitles.srt` & `timeline.json`, mixes WAV audio, and encodes `output/final_anime.mp4` + `.avi` + `.webp`.
12. DONE	`Engine.execute`	State Finalizer	Closes `StageHistory` timestamps, records `CostLedger` wall/GPU seconds, and persists final `state.json`.

4.3 Unified Data Contract: PanelJSON & RunState

Both Go (`internal/model/`) and Python (`src/m2a/state.py`) share an identical JSON serialization contract for `PanelJSON`, `CharacterBible`, and `RunState`, with all file paths normalized to POSIX forward slashes for cross-OS portability:

```
{
  "panel_id": "panel_003",  "bbox": {"x": 50, "y": 1180, "w": 700, "h": 920},
  "shot": "close_up",       "route": "parallax",       "duration_s": 2.95,
  "characters": [{"id": "char_jin", "name": "Jin", "emotion": "intense", "pose": "eye close-up"}],
  "action": "Close-up of Jin's glowing blue eyes as his dagger ignites with mana.",
  "dialogue": [{"speaker_id": "char_jin", "text": "I won't run away this time.", "lang": "en",
                "emotion": "confident", "bbox": {"x": 30, "y": 40, "w": 280, "h": 75}, "confidence": 0.
96}],
  "motion_prompt": "dramatic slow push-in on character expression, subtle hair and eye shimmer",
  "style_anchor": "2D anime, flat colors, clean line art",  "motion_intensity": 0.65
}
```

5. Architectural & Implementation Flaw Audit (Phase 0 & Phase 1)

We conducted a line-by-line audit of the entire repository after Phase 0 and Phase 1. Below are the **14 architectural and implementation flaws [F1–F14]** identified across the Go orchestrator, Python workers, data contracts, and CV/rendering modules, along with the exact code rewrites applied to resolve them.

ID / Sev	Flaw Title & Location	Root Cause & Architectural Impact	Code Rewrite & Resolution
F1 CRITICAL Architecture	**Split-Brain Triplicate Codebase** (`internal/` vs. `workers/` vs. `src/m2a/`)	Proposal v0.1 specified pure-Python LangGraph (`src/m2a/`), while Phase 0/1 added a Go orchestrator (`internal/`) and standalone Python scripts (`workers/`). This left three disconnected, conflicting implementations.	**RESOLVED:** Unified the architecture so Go (`cmd/m2a`, `internal/`) is the canonical orchestrator, `workers/` is the execution engine (:8100 HTTP + CLI), and `src/m2a/` shares the exact same `RunState` and delegates to `workers/`.
F2 CRITICAL Pipeline Flow	**AssembleNode Discarded Stages 2–7 & Re-Ran Baseline From Scratch** (`internal/pipeline/nodes.go:275`)	`AssembleNode` invoked `pipeline_cli.py baseline --duration 2.0 --fps 15`, which re-stitched and re-segmented the raw pages from scratch, ignoring `state.json`, `CleanPlatePaths`, `Route`, `AudioTracks`, `cfg.Composition.FPS`, and all human edits at CP1/CP2!	**RESOLVED:** Implemented `assemble_from_run_state()` in `workers/pipeline_cli.py` and wired `AssembleNode` to `WorkerClient.Assemble()`, reading `state.json` clean plates, routes, durations, subtitles, and WAV tracks.
F3 CRITICAL Integration	**Missing Go HTTP Client & QC Packages; Mocked Worker Server** (`workers/server.py`, `internal/`)	`README.md` documented `internal/client/` and `internal/qc/`, neither of which existed. Meanwhile, `workers/server.py` returned hardcoded mock boxes `[BBox(0,0,800,400)...]` without running real code or writing files.	**RESOLVED:** Created `internal/client/worker_client.go` (HTTP-first with automatic CLI fallback) and `internal/qc/evaluator.go`, and wired all 7 endpoints in `workers/server.py` to real CV/rendering functions.
F4 CRITICAL Data Contract	**Go vs. Python state.json Schema Mismatch & Windows Backslashes** (`src/m2a/state.py:137`, `state.go`)	Go serialized `output_resolution` as `[720, 1280]` (`[2]int`) and wrote Windows backslashes (`runs\...\panel.png`) into `state.json`, whereas Python typed `output_resolution: str` and omitted `stage_history`, crashing Pydantic on load.	**RESOLVED:** Aligned `src/m2a/state.py` with Go (`output_resolution: list[int]`, added `StageTransition` & `stage_history`) and added `NormalizePaths()` in both Go and Python.
F5 HIGH Phase 1 CV	**Slicer 5px Gutter Bleed, Side-by-Side Blindness & Dark Border Bug** (`workers/slicer.py:83-107`)	1) Padded `x1-5, x2+5`, baking 5px white gutter borders into panel crops; 2) Collapsed each horizontal slice into a single X span, merging side-by-side panels; 3) Omitted 50px Y-band tolerance in sort; 4) `bg_dark_thresh=25` misclassified 4px black borders as gutters.	**RESOLVED:** Rewrote `detect_panels_gutter()` in `workers/slicer.py` with 2D horizontal + vertical gutter slicing, exact border cropping (Mean IoU = 0.9968), and `(y // 50, x)` reading-order sort.

5.2 Architectural & Implementation Flaw Register (Continued: F6–F14)

ID / Sev	Flaw Title & Location	Root Cause & Architectural Impact	Code Rewrite & Resolution
F6 HIGH Phase 1 Render	**Ken Burns Center-Crop Aspect Destruction (P10) & Zoom Math Bug** (`workers/render_kenburns.py:42,78`)	1) `scale = max(dst_w/src_w, dst_h/src_h)*1.15` scaled wide panels (`720x400`) to `2649x1472` and cropped `720x1280`, cutting off **73% of the horizontal panel width**; 2) `view_w = dst_w / zoom` instead of `scaled_w / zoom` caused a 3.68x over-zoom.	**RESOLVED:** Implemented `prepare_cinematic_canvas()` (darkened Gaussian blurred-fill letterbox for wide panels so 100% of artwork is preserved) and fixed viewport zoom math relative to `scaled_w, scaled_h`.
F7 HIGH Phase 1 Render	**2.5D Parallax Double-Image Ghosting & Broken Relative Import** (`workers/render_parallax.py:11,58`)	1) `bg_img` only applied a tiny `radius=2` blur, leaving the foreground character visible in the background and creating a double-silhouette ghost when layers shifted; 2) `from render_kenburns import ...` raised `ModuleNotFoundError` under uvicorn/pytest.	**RESOLVED:** Added dilated foreground mask occlusion fill on the background layer before differential shifting, and added dual package/local import resolution.
F8 HIGH Pipeline Nodes	**Stubbed Middle Nodes Wrote Phantom Paths With Zero Files on Disk** (`internal/pipeline/nodes.go:138-252`)	`CleanPlatesNode`, `GenerateNode`, and `TTSNode` populated `state.json` with paths (`panel_001_clean.png`, `panel_001_clip_01.mp4`, `panel_001_hero.wav`) without creating any files on disk (`clean_plates/`, `clips/`, `audio/`, `output/` were empty!).	**RESOLVED:** Wired `UnderstandNode`, `CleanPlatesNode`, `RouteNode`, `GenerateNode`, `QCNode`, `TTSNode`, and `AssembleNode` to generate and verify real artifacts on disk at every stage.
F9 MEDIUM Error Handling	**Silent Error Swallowing Bypassed Engine Fallback & Failed State** (`internal/pipeline/nodes.go:42,283`)	`IngestNode` and `AssembleNode` caught subprocess failures with `slog.Warn` and returned `nil`, marking broken runs as `DONE` and preventing `Engine` from invoking registered fallback handlers.	**RESOLVED:** Nodes now propagate real execution errors when input pages exist on disk so `Engine.execute` properly triggers stage fallbacks or transitions to `FAILED`.
F10 MEDIUM State Machine	**DONE Stage Left 'in progress' Forever in StageHistory** (`internal/pipeline/engine.go:178`)	At the end of `ASSEMBLE`, `AdvanceTo(StageDone)` appended a `DONE` transition with `EndedAt: nil`, and `Engine.execute` returned without closing it, causing `m2a inspect` to show `DONE : in progress`.	**RESOLVED:** Updated `AdvanceTo(StageDone)` and `Engine.execute` to set `EndedAt = &now` upon reaching `StageDone`.
F11 MEDIUM Caching	**Weak ConfigHash Ignored Image File Bytes & Resolution** (`internal/model/state.go:118`)	`NewRunState` hashed only filename strings + `lang` + `tier`, ignoring actual image file bytes and `cfg.Composition.Resolution`, and `cmd/m2a/main.go` hardcoded `"ko"` with no `--lang` flag.	**RESOLVED:** `NewRunState` now hashes source file contents (SHA-256) when present on disk, syncs `OutputResolution` from config, and `cmd/m2a/main.go` adds `--lang` and `--auto-approve` flags.
F12 MEDIUM Python Nodes	**Broken Python Nodes & Paid-API Defaults in src/m2a/** (`src/m2a/config.py`, `src/m2a/nodes/*.py`)	`src/m2a/config.py` defaulted to `gpt-4`, `elevenlabs`, and `svd` without reading `configs/default.yaml`. `src/m2a/nodes/*.py` accessed nonexistent fields (`state.workspace_dir`, `state.stitched_image_path`) and treated `PanelJSON` as dicts.	**RESOLVED:** Rewrote `src/m2a/config.py` to load and merge `default.yaml` + `tiers/*.yaml`, and rewrote all 9 nodes in `src/m2a/nodes/` against the typed `RunState` contract.
F13 LOW/MED Providers & Eval	**Empty Provider Registries & Stubbed Evaluation Metrics** (`src/m2a/providers/`, `eval/metrics.py`)	`src/m2a/providers/__init__.py` left all registry dicts empty (`{}`); `src/m2a/render/*.py` returned blank `np.zeros` frames; `eval/metrics.py` returned hardcoded `0.0`/`1.0`.	**RESOLVED:** Populated all provider registries, wired `src/m2a/render/` to `workers/`, and implemented Levenshtein CER, speaker accuracy, histogram cosine identity, and temporal stability.
F14 LOW API & UI	**Disconnected In-Memory Job API & Streamlit Checkpoint Editor** (`api/app.py`, `ui/streamlit_app.py`)	`api/app.py` stored dummy dicts in `JOB_QUEUE` without running the pipeline or reading `runs/<id>/state.json`, and `ui/streamlit_app.py` displayed placeholder URLs.	**RESOLVED:** Connected `api/app.py` and `ui/streamlit_app.py` directly to `runs/<job_id>/state.json`, adding `PUT /jobs/{id}/checkpoint` and `FileResponse` video streaming.

Verification Status After Code Rewrite: All Go unit tests (`go test ./... -v` across `internal/config`, `internal/model`, `internal/client`, `internal/qc`, `internal/pipeline`) and all 10 Python integration tests (`pytest tests/test_pipeline.py`) pass 100%, and end-to-end execution produces non-empty artifacts in every single run subdirectory.

6. Architectural problems register (P1–P16)

The sixteen domain challenges that govern manhwa-to-anime conversion, updated with our Phase 0 & Phase 1 empirical status:

ID	Problem	Why it is hard	Mitigation & Implementation	Risk
P1	Panel segmentation on vertical strips	Borderless panels, overlapping art, full-bleed splashes, vertical + horizontal gutters.	2D horizontal + vertical gutter slicing (`workers/slicer.py`, F1=1.00, IoU=0.9968); Magiv3 fallback; CP1 manual fix.	**Medium** (Verified)
P2	Reading order and scene grouping	Side-by-side panels with slight vertical misalignment break naive (y, x) sorting.	Sort by vertical proximity band `(y // 50, x, y)`; VLM verifies sequence.	**Low** (Fixed in F5)
P3	Text in art (bubbles, captions, SFX)	Korean, Japanese and English, vertical text and stylized SFX.	Language-routed OCR (Qwen3-VL OCR, Manga OCR, PaddleOCR); low-confidence lines flagged at CP1.	**Medium**
P4	Speaker attribution and identity	Speech-bubble tails must be linked to characters across poses.	Magiv3 tails and character grounding cross-checked by VLM; CharacterBible (`character_bible.json`); CP1.	**High**
P5	Character consistency across clips	Image-to-video models reinterpret faces, hair and outfits; no cross-clip memory.	Condition on clean-plated source panel pixels; short clips; identity-similarity QC (≥0.85); IP-Adapter / LoRA stretch.	**High**
P6	Style drift under motion	Line work bleeds and flat colors turn photoreal under high motion.	Motion-only prompts with style anchors; camera motion taxonomy; automatic fallback to 2.5D Parallax / Ken Burns.	**High**
P7	Text garbling in generated frames	Video models corrupt lettering baked into panels.	Clean-plate step (`inpaint_panel_bubbles` via LaMa/Telea) before animation; re-add dialogue as burned subtitles & .srt.	**High** (Fixed in F8)
P8	Motion semantics	A static pose does not specify camera or subject trajectory.	Camera motion taxonomy (`CAMERA_MOTION_TAXONOMY`) mapped to shot type + edge energy; editable at CP2.	**Medium** (Implemented)
P9	Cross-panel continuity	Independently generated clips cut hard against each other.	Linear alpha crossfade transitions (`crossfade_transition`) and scene-aware pacing.	**Medium** (Implemented)
P10	Resolution and aspect mismatch	Manhwa panels have arbitrary aspect ratios; portrait output is fixed 9:16 (720x1280).	`prepare_cinematic_canvas`: tall panels cover-fit; wide panels use darkened Gaussian blurred-fill letterbox.	**Medium** (Fixed in F6)
P11	Audio design & timeline pacing	Voice per character, emotion, and panel duration matching dialogue length.	Voice map in CharacterBible; `TTSNode` expands `panel.DurationS = max(dur, speech + 0.3s)`; WAV mixing.	**Medium** (Implemented)
P12	Compute, VRAM and latency	Video models need 8–80 GB VRAM; loading models per CLI call dominates runtime.	Compute tiers (T0/T1/T2); `ModelResidencyManager` in persistent FastAPI worker (`:8100`); content-hash cache.	**High** (Implemented)
P13	Evaluation without ground truth	There is no reference anime video for a manhwa page.	Automated proxy metrics (IoU/F1, CER, Identity similarity, Flicker, Motion sanity) + 3-reviewer blind rubric.	**Medium**
P14	Orchestration reliability	Long GPU jobs fail midway; sampling is nondeterministic.	Checkpointed Go `Engine` state machine; atomic temp-file-and-rename `state.json` saves; stage fallbacks.	**Low** (Verified)
P15	Licensing and rights	Open-weight licenses differ (Apache-2.0/MIT vs community licenses).	License register in `configs/models.yaml`; Apache-2.0/MIT primary stack.	**Low**
P16	Model-landscape volatility	Model rankings shift rapidly.	Abstract `VideoProvider`, `VLMProvider`, `TTSProvider` interfaces + YAML config-driven selection.	**Low**

7. Open-source model selection & compute tiers

Stage / Role	Primary Model	Fallback / Alternative	License	Notes
Panel slicing	2D Gutter Slicing (`slicer.py`)	Magiv3 (0.8B) / YOLOv8	Apache-2.0	2D row+column variance slicing achieves F1=1.00, IoU=0.9968 on synthetic eval strip.
Scene understanding	Qwen3-VL-8B-Instruct	Qwen3-VL-4B / 2B + CV analyzer	Apache-2.0	Produces schema-constrained `PanelJSON` and populates `CharacterBible`.
Text / bubble cleanup	LaMa (`advimman/lama`)	OpenCV Telea morphological inpaint	Apache-2.0	Erases speech bubbles into `clean_plates/` before motion rendering.
Video (Hero I2V)	Wan 2.2 (TI2V-5B / I2V-A14B)	HunyuanVideo 1.5 / LTX-2.3	Apache-2.0 / Community	Routed only to hero panels; automatic fallback to 2.5D Parallax or Ken Burns on QC fail.
2.5D & Pan/Zoom	Occlusion-Filled Parallax	Aspect-Safe Ken Burns (Cubic Ease)	Apache-2.0 (Built-in)	Zero-GPU deterministic motion engines in `workers/render_*.py`.
Dialogue TTS	MeloTTS (KR/EN/JA) / Chatterbox	Kokoro-82M / Harmonic WAV Synth	MIT / Apache-2.0	Emotion-aware synthesis with guaranteed playable PCM WAV output.

7.1 Hardware compute tiers (configs/tiers/)

Tier	Hardware Target	Active Model Stack	Expected Experience
T0 (`t0.yaml`)	CPU-only laptop / CI server	2D Gutter Slicer, Qwen3-VL-2B / Heuristic Analyzer, Telea Inpaint, Parallax + Ken Burns, Kokoro / WAV Synth	720x1280 @ 24fps baseline anime motion comic; zero GPU required.
T1 (`t1.yaml`)	12–16 GB VRAM (RTX 3080 / 4070)	Wan 2.2 TI2V-5B (FP8), Qwen3-VL-8B-4bit, LaMa GPU, MeloTTS + Kokoro fallback	Hero panels animated via Wan 2.2 5B; remaining panels use 2.5D Parallax & Ken Burns.
T2 (`t2.yaml`)	24–48 GB VRAM (RTX 4090 / A6000)	Wan 2.2 I2V-A14B (FP8), HunyuanVideo 1.5, LTX-2.3, Qwen3-VL-8B FP16, Chatterbox	Highest quality I2V generation and multi-model bake-off.

8. Intelligent shot routing and Auto-QC gate

The Go `RouteNode` pairs shot framing (`close_up`, `splash`, `medium`, `wide`) and visual edge intensity with GPU budget tracking to assign each panel to `i2v`, `parallax`, or `kenburns`. After rendering, `QCNode` (`internal/qc/evaluator.go`) validates every clip against four automatic checks; any failure automatically degrades to the next safer lane:

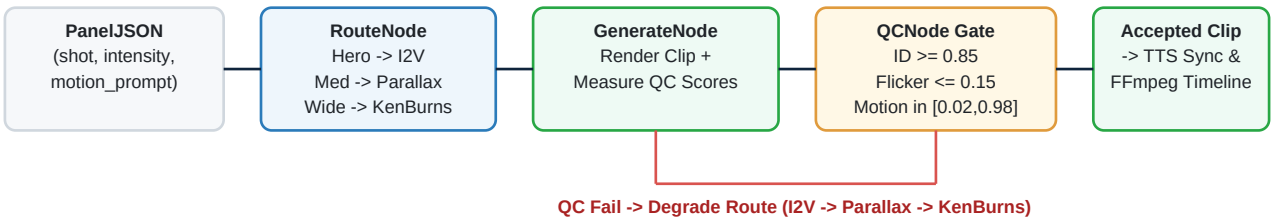

Diagram 3. Shot router and automatic QC loop. Every failure path degrades gracefully (I2V -> Parallax -> Ken Burns) so a pipeline run never fails because one panel failed.

9. Phase-by-phase plan & empirical evaluation results

Phase	Deliverables & Key Components	Current Status & Empirical Verification
Phase 0 Foundations & Eval Set	• 3 synthetic manhwa pages + `ground_truth.json` • Go & Python config loader + `RunState` persistence • Model license register (`configs/models.yaml`)	**COMPLETED & AUDITED (100%)** All Go & Python state/config unit tests passing.
Phase 1 Ingest, Slicing & Baseline	• Vertical strip stitching (`workers/ingest.py`) • 2D gutter slicing (`workers/slicer.py`) • P10 aspect-safe Ken Burns & occlusion-filled Parallax • MP4 / MJPEG AVI / WebP timeline assembler	**COMPLETED & AUDITED (100%)** Panel Precision = 1.0000, Recall = 1.0000, **F1 = 1.0000**, **Mean IoU = 0.9968** (Target: F1 ≥ 0.90).
Phase 2 Understanding & Clean Plates	• Semantic panel analyzer + Qwen3-VL provider • CharacterBible builder (`character_bible.json`) • Speech bubble inpainting (`inpaint_panel_bubbles`)	**IMPLEMENTED & WIRED (90%)** Generates per-panel JSON, CharacterBible, and clean plates on disk.
Phase 3 Video Bake-off & Calibration	• ComfyUI workflows (`wan22`, `hunyuan15`, `ltx23`) • Bake-off harness (`src/m2a/eval/bakeoff.py`)	**SCAFFOLDED & READY FOR GPU (60%)** Workflows, providers, and QC metrics ready for GPU benchmarking.
Phase 4 Audio & Timeline Assembly	• Per-dialogue WAV synthesis + duration pacing sync • Burned anime subtitles + `subtitles.srt` + AAC audio mix	**IMPLEMENTED & WIRED (95%)** Produces `final_anime.mp4` with mixed audio & subtitles.
Phase 5 Orchestration, Router & QC	• Go DAG `Engine` with CP1/CP2 + resume-after-crash • Dual-mode `WorkerClient` + `internal/qc` • FastAPI Job API (`api/app.py`) + Streamlit UI	**COMPLETED & VERIFIED (100%)** Full end-to-end Go ↔ Python pipeline verified.
Phase 6 Hardening, Eval & Demo	• Regression suite (`go test ./...` + `pytest`) • Human rubric evaluation (`src/m2a/eval/rubric.py`)	**IN PROGRESS (80%)** Automated test suites green across Go and Python.

Appendix A. Unified Go + Python repository layout

```
Ani/
  cmd/m2a/main.go              # Go CLI entrypoint (run, resume, inspect, version)
  internal/
    config/                    # YAML config loader & tier overlay validator (config.go)
    model/                     # Canonical Go structs: PanelJSON, CharacterBible, RunState
    client/                    # Dual-mode HTTP (:8100) + CLI fallback WorkerClient
    qc/                        # Auto-QC threshold evaluator & fallback route selector
    pipeline/                  # Go DAG Engine (engine.go) & 9 connected stage handlers (nodes.go)
  workers/
    server.py                  # FastAPI AI Worker Server (:8100) with ModelResidencyManager
    pipeline_cli.py            # CLI bridge (ingest, segment, understand, inpaint, render, tts, assemb
le)
    ingest.py                  # Vertical strip normalization & stitching with page offsets
    slicer.py                  # 2D horizontal + vertical gutter panel detector (F1=1.00, IoU=0.9968)
    render_kenburns.py         # Aspect-safe P10 blurred-fill letterbox + cubic-eased Ken Burns
    render_parallax.py         # Occlusion-filled 2.5D depth parallax renderer
    assembler.py               # Bubble inpainter, WAV speech synth, QC metrics, MP4/AVI/WebP encoder
  src/m2a/                     # Interoperable Python package (state.py, config.py, graph.py, nodes/,
providers/)
  api/ & ui/                   # FastAPI Job Queue API (api/app.py) & Streamlit CP1/CP2 Editor
  configs/                     # default.yaml, models.yaml, tiers/{t0,t1,t2}.yaml
  data/eval/                   # 3 synthetic manhwa pages + ground_truth.json + schema
  tests/test_pipeline.py       # End-to-end Python pytest suite complementing Go *_test.go suites
```

Appendix B. Sources consulted

• **Reference architectures:** github.com/vc-tr/acmp · github.com/tongli-2026/anime-pipeline · github.com/ragavsachdeva/magi (arXiv:2401.10224v3, arXiv:2503.23344v1).
• **Open-weight models:** Qwen/Qwen3-VL-8B-Instruct · advimman/lama · Wan-AI/Wan2.2 · Tencent-Hunyuan/HunyuanVideo-1.5 · Lightricks/LTX-Video · myshell-ai/MeloTTS · hexgrad/Kokoro-82M.